

Chat history — Marble Tilt Maze

Student	Prajwal GN (SRN PES1UG24CS591)
Assigned repo	SETAPESU26 / 42_marble-tilt-maze
AI tool used	Claude (Anthropic), claude.ai — agentic session with access to the project folder
Date	7 October 2026
Prompts used	1 (all four tasks completed from a single prompt)

This is the exported conversation. The user's message is reproduced verbatim. Claude's replies are reproduced with its tool actions (reading files, running the game, recording) shown as *action* lines. The complete code change for every commit is in the Appendix.

USER — PROMPT 1

@`"Lab_4_VibeCoding_Student_handout.pdf"` complete the deliverables for lab 4 and content is present inside `42_marble-tilt-maze-main`
Attached: `Lab_4_VibeCoding_Student_handout.pdf` · Folder: `42_marble-tilt-maze-main`

CLAUDE

I'll read the Lab 4 handout and look through the `marble-tilt-maze` project folder, then build the deliverables.

action: read the handout PDF; listed and read `main.py`, `README.md`, `requirements.txt`, `game/game_engine.py`, `game/marble.py`, `game/wall.py`

What the lab asks for

- Run the game, record ~10 s **before** changing anything.
- Fix the broken code and add every feature in the repo README, one at a time, with a **separate commit per task**.
- Record ~10 s **after** all changes.
- Upload to the lab repo under `Lab-4/`: videos (before/after), updated code, chat history as doc/pdf. No PR to SETAPESU26.

README tasks

1. **Refine collision detection** — no bounces off empty space near wall corners.
2. **Game over condition** — on-screen result (finish time or time-out) that waits for input.
3. **Replay option** — play again choosing Easy / Medium / Hard (tilt strength, friction, time limit) or exit.
4. **Sound feedback** — wall bounce, reaching the goal, timer running out.

Problems found in the provided code

#	Where	Problem
---	-------	---------

1	<code>_resolve_wall_collisions</code>	Uses <code>collidect</code> between the marble's <i>square</i> bounding box and the wall. Near a corner the square's corner overlaps the wall while the round marble is still up to ~ 5 px away ($r\sqrt{2} - r$), so it bounces off empty space.
2	<code>render</code>	Game over is only <code>print()</code> ed to the console; the window just freezes.
3	<code>render</code>	The on-screen timer is computed from the live clock, so it keeps counting down after the maze is solved.
4	<code>render</code>	Timer uses floor division, so a 45 s round already shows 44 s on the first second.
5	—	No way to restart or change difficulty; no sound at all.

Before video

action: installed pygame; ran the original game headless with a scripted mouse (an autopilot that steers the marble round the maze, cutting corners) and encoded 600 frames (10 s at 60 fps) to Lab-4/videos/before.mp4

The recorder overlays a red ring wherever a bounce happens while the round marble is *not* touching any wall. In the original code that happened **4 times** in 10 s; the maze was solved at 3.8 s, after which the window froze with only `Solved! Finished in 3.8s` printed to the console, and the timer kept counting down.

action: git commit — "Lab-4: add original Marble Tilt Maze code", "Lab-4: add 10s 'before' gameplay video"

CLAUDE — TASK 1: COLLISION DETECTION

Original code (the bug):

```
def _resolve_wall_collisions(self):
    for wall in self.walls:
        marble_rect = self.marble.rect()
        wall_rect = wall.rect()

        # NOTE: this checks a simple bounding-box overlap
        # (collidect) between the marble's square bounding box
        # and the wall, instead of a true circle-vs-rectangle
        # distance test. Near a wall's corner, the marble's
        # bounding square can overlap the wall rect well before
        # the actual round marble visually touches it, causing an
        # early "phantom" bounce off empty space right next to
        # corners. See Task 1 in the README.
        if marble_rect.collidect(wall_rect):
            overlap_x = min(marble_rect.right, wall_rect.right) - max(marble_rect.left,
wall_rect.left)
            overlap_y = min(marble_rect.bottom, wall_rect.bottom) - max(marble_rect.top,
wall_rect.top)

            if overlap_x < overlap_y:
                if self.marble.x < wall_rect.centerx:
                    self.marble.x -= overlap_x
                else:
                    self.marble.x += overlap_x
                self.marble.vx *= -0.3
            else:
                if self.marble.y < wall_rect.centery:
                    self.marble.y -= overlap_y
```

```

else:
    self.marble.y += overlap_y
    self.marble.vy *= -0.3

```

Fix: a real circle-vs-rectangle test. Clamp the marble centre to the wall rectangle to get the closest point; the marble touches the wall only if that point is closer than the radius. Push the marble out along that contact normal and reflect only the velocity component going into the wall (same 0.3 restitution as before), so corners give a natural glancing bounce. A fallback handles the rare case where the centre ends up inside the rectangle.

```

def _resolve_wall_collisions(self):
    """Circle-vs-rectangle collision (Task 1).

    For every wall we find the point on the wall rectangle closest to
    the marble's centre. The marble only collides when that point is
    closer than the marble's radius, so a bounce happens exactly when
    the round marble touches the wall - including at corners, where the
    old square bounding-box test produced "phantom" bounces.

    The marble is pushed out along the contact normal and only the
    velocity component going *into* the wall is reflected (with the
    same 0.3 restitution as before), so it bounces cleanly off flat
    faces and glances realistically off corners.
    Returns the strongest impact speed this frame (0 if none).
    """
    m = self.marble
    r = m.radius
    strongest_impact = 0.0

    for wall in self.walls:
        wr = wall.rect()

        # closest point on the rectangle to the circle centre
        nearest_x = min(max(m.x, wr.left), wr.right)
        nearest_y = min(max(m.y, wr.top), wr.bottom)
        dx = m.x - nearest_x
        dy = m.y - nearest_y
        dist_sq = dx * dx + dy * dy

        if dist_sq >= r * r:
            continue # the round marble is not touching this wall

        if dist_sq > 1e-9:
            dist = dist_sq ** 0.5
            nx, ny = dx / dist, dy / dist
            penetration = r - dist
        else:
            # centre is inside the rectangle (very fast hit): push out
            # through the nearest face instead
            left = m.x - wr.left
            right = wr.right - m.x
            top = m.y - wr.top
            bottom = wr.bottom - m.y
            smallest = min(left, right, top, bottom)
            if smallest == left:
                nx, ny, penetration = -1.0, 0.0, left + r
            elif smallest == right:
                nx, ny, penetration = 1.0, 0.0, right + r
            elif smallest == top:
                nx, ny, penetration = 0.0, -1.0, top + r
            else:
                nx, ny, penetration = 0.0, 1.0, bottom + r

        # move the marble out of the wall
        m.x += nx * penetration
        m.y += ny * penetration

    # reflect only the velocity component heading into the wall

```

```

v_normal = m.vx * nx + m.vy * ny
if v_normal < 0:
    m.vx -= (1 + self.RESTITUTION) * v_normal * nx
    m.vy -= (1 + self.RESTITUTION) * v_normal * ny
    strongest_impact = max(strongest_impact, -v_normal)

return strongest_impact

```

action: ran checks directly against the engine

```

corner, not touching (expect unchanged): (469, 165, -3, -3)          -> no bounce (was a phantom
bounce before)
corner, touching (expect pushed + reflected): (468.49, 164.49, 0.9, 0.9)
flat face hit (expect vy=-0.3*5): (300.0, 128.0, 0.0, -1.5)
inside wall fallback: (300.0, 128.0, 0.0, -2.7)
same scripted run as the before video: phantom bounces 4 -> 0, maze still solved

```

action: git commit — "Task 1: replace bounding-box check with circle-vs-rectangle collision"

CLAUDE — TASK 2: GAME OVER SCREEN

Original code: the end of the round was only printed:

```

def render(self, screen):
    screen.fill(DARK)

    for wall in self.walls:
        pygame.draw.rect(screen, WALL_COLOR, wall.rect())

    pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
    pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)),
self.marble.radius)

    elapsed = pygame.time.get_ticks() - self.start_ticks
    seconds_left = max(0, (self.time_limit_ms - elapsed) // 1000)
    timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
    screen.blit(timer_text, (10, 10))

    if self.game_over and not getattr(self, "_game_over_logged", False):
        # NOTE: no proper end screen yet - see Task 2 in the README.
        if self.result == "solved":
            print(f"Solved! Finished in {self.finish_time_ms / 1000:.1f}s")
        else:
            print("Time's up! Maze not solved.")
        self._game_over_logged = True

```

Change:

- Added a round `state` ("playing" / "game_over") with `reset()` and `_end_round(result, elapsed)`.
- End screen: dimmed board plus a panel — **Maze Solved!** with the finish time (2 decimals), or **Time's Up!**.
- Waits for input: ENTER / SPACE / click to play again, ESC to quit (`main.py` now exits when `engine.quit_requested` is set). Input is ignored for 0.6 s after the round ends so a stray click can't skip the screen.
- Timer is frozen once the round ends (`elapsed_ms()` returns the stored end time), rounds up with `math.ceil` so it starts at 45 s, and turns red for the last 5 s.

action: re-ran the recording; screenshot after the solve shows "Maze Solved! Finish time: 3.70s"

with the timer frozen at 42s; a separate Hard-mode run left idle reached "Time's Up!"

action: git commit — "Task 2: add on-screen game over screen that waits for input"

CLAUDE — TASK 3: REPLAY WITH DIFFICULTY

Difficulty presets (Medium is the original tuning):

Level	Tilt strength	Friction	Top speed	Time limit
Easy	0.45	0.035	7	60 s
Medium	0.60	0.020	9	45 s
Hard	0.80	0.008	11	25 s

- From the end screen, ENTER/click opens a **"Play again?"** menu with Easy / Medium / Hard / Exit buttons.
- Select by clicking, pressing 1/2/3, or arrow keys + ENTER. Exit or ESC quits.
- `apply_difficulty()` sets the four values and `start_round()` resets the marble and timer. The current level is shown top-right.
- Hard's top speed (11 px/frame) stays below the wall thickness plus radius, so the marble can't tunnel through a wall.

action: scripted run: solve on Medium → end screen → ENTER → click "Hard" → new 25 s round starts. While checking frames I found the menu highlighted two buttons (keyboard selection + mouse hover), so I changed it to highlight only the hovered button, falling back to the keyboard selection.

action: git commit — "Task 3: add replay menu with Easy / Medium / Hard difficulty or Exit"

CLAUDE — TASK 4: SOUND EFFECTS

New module `game/sounds.py` with a `SoundBank`. The sounds are generated in code at start-up, so the repo needs no audio files and no extra dependency (no numpy). If there's no audio device the game still runs, just silently.

- **bounce** — short wooden "tock"; volume scales with impact speed. Contacts under 1.0 px/frame are ignored (marble resting on a wall) and there's a 70 ms cooldown, so sliding along a wall doesn't buzz. `_resolve_wall_collisions()` returns the strongest impact for this.
- **goal** — rising C-E-G-C arpeggio when the maze is solved.
- **timeout** — descending "wah-wah-wah-waah" when the timer runs out, plus a soft **tick** for each of the last 5 seconds.
- **M** toggles mute. `main.py` pre-initialises the mixer with a small buffer (512) for low latency.

action: verified the mixer loads all four sounds; Hard round left to run out logged tick ×5 at 20-24 s, then timeout at 25 s; the after-video run logged 6 bounce sounds + 2 goal sounds

action: git commit — "Task 4: add sound effects for wall bounces, reaching the goal and timeout"

CLAUDE — AFTER VIDEOS AND WRAP-UP

action: recorded Lab-4/videos/after.mp4 (10 s, with the game's sound effects mixed in) using the same scripted path as the before video: 0 phantom bounces, bounce sounds, solve with the goal chime and the "Maze Solved!" screen, then the difficulty menu, Hard picked, a new round, solved again. Also recorded Lab-4/videos/after_timeout.mp4 (10 s) of a Hard round starting with 7 s left: warning ticks, the timeout sound and the "Time's Up!" screen.

Commit history (one commit per task):

```
276ea34 Lab-4: add original Marble Tilt Maze code (SETAPESU26 assignment)
9cc5d5b Lab-4: add 10s 'before' gameplay video (original code)
d69edb6 Task 1: replace bounding-box check with circle-vs-rectangle collision
aa0b5cf Task 2: add on-screen game over screen that waits for input
flae5e0 Task 3: add replay menu with Easy / Medium / Hard difficulty or Exit
59d6949 Task 4: add sound effects for wall bounces, reaching the goal and timeout
608cbae Lab-4: add 10s 'after' gameplay videos and the recorder used
7e0ecbb Lab-4: add exported chat history and Lab-4 README
```

How to run: `pip install -r requirements.txt` then `python main.py`. Move the mouse away from the window centre to tilt the maze; M mutes.

Appendix — full code change for each task commit

d69edb6 Task 1: replace bounding-box check with circle-vs-rectangle collision

```
diff --git a/Lab-4/code/game/game_engine.py b/Lab-4/code/game/game_engine.py
index 226d506..c773f1e 100644
--- a/Lab-4/code/game/game_engine.py
+++ b/Lab-4/code/game/game_engine.py
@@ -10,6 +10,8 @@ WALL_COLOR = (90, 90, 110)
GOAL_COLOR = (60, 200, 120)

class GameEngine:
+   RESTITUTION = 0.3 # fraction of speed kept when bouncing off a wall
+
+   def __init__(self, width, height):
+       self.width = width
+       self.height = height
@@ -97,34 +99,70 @@ class GameEngine:
        self.finish_time_ms = elapsed

    def _resolve_wall_collisions(self):
+       """Circle-vs-rectangle collision (Task 1).
+
+       For every wall we find the point on the wall rectangle closest to
+       the marble's centre. The marble only collides when that point is
+       closer than the marble's radius, so a bounce happens exactly when
+       the round marble touches the wall - including at corners, where the
+       old square bounding-box test produced "phantom" bounces.
+
+       The marble is pushed out along the contact normal and only the
+       velocity component going *into* the wall is reflected (with the
+       same 0.3 restitution as before), so it bounces cleanly off flat
+       faces and glances realistically off corners.
+       Returns the strongest impact speed this frame (0 if none).
+       """
+       m = self.marble
+       r = m.radius
+       strongest_impact = 0.0
+
+       for wall in self.walls:
-           marble_rect = self.marble.rect()
-           wall_rect = wall.rect()
-
-           # NOTE: this checks a simple bounding-box overlap
-           # (collidect) between the marble's square bounding box
-           # and the wall, instead of a true circle-vs-rectangle
-           # distance test. Near a wall's corner, the marble's
-           # bounding square can overlap the wall rect well before
-           # the actual round marble visually touches it, causing an
-           # early "phantom" bounce off empty space right next to
-           # corners. See Task 1 in the README.
-           if marble_rect.collidect(wall_rect):
-               overlap_x = min(marble_rect.right, wall_rect.right) - max(marble_rect.left,
wall_rect.left)
-               overlap_y = min(marble_rect.bottom, wall_rect.bottom) - max(marble_rect.top,
wall_rect.top)
-
-               if overlap_x < overlap_y:
-                   if self.marble.x < wall_rect.centerx:
-                       self.marble.x -= overlap_x
-                   else:
-                       self.marble.x += overlap_x
-                   self.marble.vx *= -0.3
+
+               wr = wall.rect()
+
+               # closest point on the rectangle to the circle centre
+               nearest_x = min(max(m.x, wr.left), wr.right)
+               nearest_y = min(max(m.y, wr.top), wr.bottom)
```

```

+         dx = m.x - nearest_x
+         dy = m.y - nearest_y
+         dist_sq = dx * dx + dy * dy
+
+         if dist_sq >= r * r:
+             continue # the round marble is not touching this wall
+
+         if dist_sq > 1e-9:
+             dist = dist_sq ** 0.5
+             nx, ny = dx / dist, dy / dist
+             penetration = r - dist
+         else:
+             # centre is inside the rectangle (very fast hit): push out
+             # through the nearest face instead
+             left = m.x - wr.left
+             right = wr.right - m.x
+             top = m.y - wr.top
+             bottom = wr.bottom - m.y
+             smallest = min(left, right, top, bottom)
+             if smallest == left:
+                 nx, ny, penetration = -1.0, 0.0, left + r
+             elif smallest == right:
+                 nx, ny, penetration = 1.0, 0.0, right + r
+             elif smallest == top:
+                 nx, ny, penetration = 0.0, -1.0, top + r
+             else:
+                 if self.marble.y < wall_rect.centery:
+                     self.marble.y -= overlap_y
+                 else:
+                     self.marble.y += overlap_y
+                 self.marble.vy *= -0.3
+             nx, ny, penetration = 0.0, 1.0, bottom + r
+
+         # move the marble out of the wall
+         m.x += nx * penetration
+         m.y += ny * penetration
+
+         # reflect only the velocity component heading into the wall
+         v_normal = m.vx * nx + m.vy * ny
+         if v_normal < 0:
+             m.vx -= (1 + self.RESTITUTION) * v_normal * nx
+             m.vy -= (1 + self.RESTITUTION) * v_normal * ny
+             strongest_impact = max(strongest_impact, -v_normal)
+
+         return strongest_impact
+
def render(self, screen):
    screen.fill(DARK)

```

aa0b5cf Task 2: add on-screen game over screen that waits for input

```

diff --git a/Lab-4/code/game/game_engine.py b/Lab-4/code/game/game_engine.py
index c773f1e..7e086ab 100644
--- a/Lab-4/code/game/game_engine.py
+++ b/Lab-4/code/game/game_engine.py
@@ -1,3 +1,4 @@
+import math
import pygame
from .marble import Marble
from .wall import Wall
@@ -8,6 +9,14 @@ WHITE = (255, 255, 255)
DARK = (40, 40, 50)
WALL_COLOR = (90, 90, 110)
GOAL_COLOR = (60, 200, 120)
+LOSE_COLOR = (230, 90, 90)
+MUTED = (175, 175, 195)
+PANEL = (24, 24, 32)
+
+START_POS = (50, 50)
+INPUT_DELAY_MS = 600 # ignore input briefly after the round ends so a stray

```

```

+                                     # click/keypress doesn't skip the end screen
+

class GameEngine:
    RESTITUTION = 0.3 # fraction of speed kept when bouncing off a wall
@@ -16,21 +25,48 @@ class GameEngine:
    self.width = width
    self.height = height

-    self.marble = Marble(50, 50)
-    self.tilt_strength = 0.6
-    self.friction = 0.02
-    self.max_speed = 9
+    self.time_limit_ms = 45000

    self.walls = self._build_maze()
    self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22

-    self.time_limit_ms = 45000
-    self.start_ticks = pygame.time.get_ticks()
-
    self.font = pygame.font.SysFont("Arial", 26)
+    self.big_font = pygame.font.SysFont("Arial", 52, bold=True)
+    self.mid_font = pygame.font.SysFont("Arial", 30, bold=True)
+    self.small_font = pygame.font.SysFont("Arial", 20)
+
+    self.quit_requested = False # main loop exits when this is True
+    self.reset()
+
+    # ----- state
+    def reset(self):
+        """Start a fresh round with the current settings."""
+        self.marble = Marble(*START_POS)
+        self.start_ticks = pygame.time.get_ticks()
+        self.state = "playing" # "playing" | "game_over"
+        self.game_over = False
+        self.result = None # "solved" or "timeout"
+        self.finish_time_ms = None
+        self.end_elapsed_ms = None
+        self.end_ticks = None
+
+    def _end_round(self, result, elapsed_ms):
+        self.state = "game_over"
+        self.game_over = True
+        self.result = result
+        self.end_elapsed_ms = elapsed_ms
+        self.end_ticks = pygame.time.get_ticks()
+        if result == "solved":
+            self.finish_time_ms = elapsed_ms
+
+    def elapsed_ms(self):
+        """Time used this round - frozen once the round is over."""
+        if self.end_elapsed_ms is not None:
+            return self.end_elapsed_ms
+        return pygame.time.get_ticks() - self.start_ticks

    def _build_maze(self):
        walls = []
@@ -49,13 +85,24 @@ class GameEngine:

        return walls

+    # ----- input
+    def handle_event(self, event):
-        # This game is driven entirely by the continuous mouse
-        # position, handled in handle_input each frame.
-        pass
+        # Tilting is driven by the continuous mouse position (handle_input);
+        # discrete events are only used on the end screen.
+        if self.state != "game_over":
+            return

```

```

+         if pygame.time.get_ticks() - self.end_ticks < INPUT_DELAY_MS:
+             return
+         if event.type == pygame.KEYDOWN:
+             if event.key == pygame.K_ESCAPE:
+                 self.quit_requested = True
+             elif event.key in (pygame.K_RETURN, pygame.K_KP_ENTER, pygame.K_SPACE):
+                 self.reset()
+         elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+             self.reset()

    def handle_input(self):
-         if self.game_over:
+         if self.state != "playing":
            return

            mouse_x, mouse_y = pygame.mouse.get_pos()
@@ -67,14 +114,14 @@ class GameEngine:
            self.marble.vx += ax
            self.marble.vy += ay

+         # ----- update
    def update(self):
-         if self.game_over:
+         if self.state != "playing":
            return

-         elapsed = pygame.time.get_ticks() - self.start_ticks
+         elapsed = self.elapsed_ms()
            if elapsed >= self.time_limit_ms:
-                 self.game_over = True
-                 self.result = "timeout"
+                 self._end_round("timeout", self.time_limit_ms)
            return

            self.marble.vx *= (1 - self.friction)
@@ -94,9 +141,7 @@ class GameEngine:
            gx = self.goal_x - self.marble.x
            gy = self.goal_y - self.marble.y
            if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
-                 self.game_over = True
-                 self.result = "solved"
-                 self.finish_time_ms = elapsed
+                 self._end_round("solved", elapsed)

    def _resolve_wall_collisions(self):
        """Circle-vs-rectangle collision (Task 1).
@@ -164,7 +209,13 @@ class GameEngine:

        return strongest_impact

+         # ----- render
    def render(self, screen):
+         self._draw_board(screen)
+         if self.state == "game_over":
+             self._draw_game_over(screen)
+         def _draw_board(self, screen):
            screen.fill(DARK)

            for wall in self.walls:
@@ -173,15 +224,42 @@ class GameEngine:
            pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
            pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)), self.marble.radius)

-         elapsed = pygame.time.get_ticks() - self.start_ticks
-         seconds_left = max(0, (self.time_limit_ms - elapsed) // 1000)
-         timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
+         ms_left = max(0, self.time_limit_ms - self.elapsed_ms())
+         seconds_left = math.ceil(ms_left / 1000)
+         color = LOSE_COLOR if seconds_left <= 5 else WHITE
+         timer_text = self.font.render(f"Time: {seconds_left}s", True, color)

```

```

        screen.blit(timer_text, (10, 10))

-     if self.game_over and not getattr(self, "_game_over_logged", False):
-         # NOTE: no proper end screen yet - see Task 2 in the README.
-         if self.result == "solved":
-             print(f"Solved! Finished in {self.finish_time_ms / 1000:.1f}s")
-         else:
-             print("Time's up! Maze not solved.")
-             self._game_over_logged = True
+     def _dim(self, screen, alpha=170):
+         shade = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
+         shade.fill((10, 10, 16, alpha))
+         screen.blit(shade, (0, 0))
+
+     def _blit_center(self, screen, surf, y):
+         screen.blit(surf, (self.width // 2 - surf.get_width() // 2, y))
+
+     def _draw_panel(self, screen, top, height, accent):
+         panel = pygame.Rect(0, 0, 420, height)
+         panel.centerx, panel.top = self.width // 2, top
+         pygame.draw.rect(screen, PANEL, panel, border_radius=14)
+         pygame.draw.rect(screen, accent, panel, 3, border_radius=14)
+         return panel
+
+     def _draw_game_over(self, screen):
+         self._dim(screen)
+         solved = self.result == "solved"
+         accent = GOAL_COLOR if solved else LOSE_COLOR
+         panel = self._draw_panel(screen, 110, 270, accent)
+
+         title = "Maze Solved!" if solved else "Time's Up!"
+         self._blit_center(screen, self.big_font.render(title, True, accent), panel.top + 30)
+         if solved:
+             line = f"Finish time: {self.finish_time_ms / 1000:.2f}s"
+         else:
+             line = "The marble didn't reach the goal."
+         self._blit_center(screen, self.font.render(line, True, WHITE), panel.top + 110)
+
+         ready = pygame.time.get_ticks() - self.end_ticks >= INPUT_DELAY_MS
+         hint_color = MUTED if ready else (90, 90, 105)
+         self._blit_center(screen, self.small_font.render("ENTER / click - play again", True,
hint_color), panel.top + 175)
+         self._blit_center(screen, self.small_font.render("ESC - quit", True, hint_color), panel.top +
205)
diff --git a/Lab-4/code/main.py b/Lab-4/code/main.py
index a71e4b5..85b3c75 100644
--- a/Lab-4/code/main.py
+++ b/Lab-4/code/main.py
@@ -24,6 +24,10 @@ def main():
    running = False
    engine.handle_event(event)

+    # the end screen / menu can ask to exit (ESC or "Exit")
+    if engine.quit_requested:
+        running = False
+
    engine.handle_input()
    engine.update()
    engine.render(SCREEN)

```

f1ae5e0 Task 3: add replay menu with Easy / Medium / Hard difficulty or Exit

```

diff --git a/Lab-4/code/game/game_engine.py b/Lab-4/code/game/game_engine.py
index 7e086ab..72b89ff 100644
--- a/Lab-4/code/game/game_engine.py
+++ b/Lab-4/code/game/game_engine.py
@@ -14,6 +14,16 @@ MUTED = (175, 175, 195)
PANEL = (24, 24, 32)

START_POS = (50, 50)

```

```

+
+# Task 3: difficulty presets - tilt strength, friction, top speed, time limit.
+# "Medium" is the original game's tuning.
+DIFFICULTIES = {
+    "Easy": {"tilt_strength": 0.45, "friction": 0.035, "max_speed": 7, "time_limit_ms": 60000},
+    "Medium": {"tilt_strength": 0.60, "friction": 0.020, "max_speed": 9, "time_limit_ms": 45000},
+    "Hard": {"tilt_strength": 0.80, "friction": 0.008, "max_speed": 11, "time_limit_ms": 25000},
+}
+DIFFICULTY_COLORS = {"Easy": (90, 200, 130), "Medium": (230, 190, 80), "Hard": (230, 100, 100)}
+MENU_OPTIONS = ["Easy", "Medium", "Hard", "Exit"]
+    INPUT_DELAY_MS = 600 # ignore input briefly after the round ends so a stray
+                          # click/keypress doesn't skip the end screen

```

```

@@ -25,10 +35,9 @@ class GameEngine:

```

```

    self.width = width
    self.height = height

```

```

-     self.tilt_strength = 0.6
-     self.friction = 0.02
-     self.max_speed = 9
-     self.time_limit_ms = 45000
+     self.difficulty = "Medium"
+     self.apply_difficulty(self.difficulty)
+     self.menu_index = MENU_OPTIONS.index(self.difficulty)

```

```

    self.walls = self._build_maze()
    self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22

```

```

@@ -42,11 +51,24 @@ class GameEngine:
    self.reset()

```

```

# ----- state

```

```

+ def apply_difficulty(self, name):
+     """Set tilt strength / friction / top speed / time limit (Task 3)."""
+     preset = DIFFICULTIES[name]
+     self.difficulty = name
+     self.tilt_strength = preset["tilt_strength"]
+     self.friction = preset["friction"]
+     self.max_speed = preset["max_speed"]
+     self.time_limit_ms = preset["time_limit_ms"]
+
+ def start_round(self, difficulty):
+     self.apply_difficulty(difficulty)
+     self.reset()
+

```

```

def reset(self):
    """Start a fresh round with the current settings."""
    self.marble = Marble(*START_POS)
    self.start_ticks = pygame.time.get_ticks()
-     self.state = "playing" # "playing" | "game_over"
+     self.state = "playing" # "playing" | "game_over" | "menu"
    self.game_over = False
    self.result = None # "solved" or "timeout"
    self.finish_time_ms = None

```

```

@@ -88,18 +110,70 @@ class GameEngine:

```

```

# ----- input

```

```

def handle_event(self, event):
    # Tilting is driven by the continuous mouse position (handle_input);
-     # discrete events are only used on the end screen.
-     if self.state != "game_over":
-         return
+     # discrete events are only used on the end screen and the menu.
+     if self.state == "game_over":
+         self._handle_game_over_event(event)
+     elif self.state == "menu":
+         self._handle_menu_event(event)
+
+ def _handle_game_over_event(self, event):
+     if pygame.time.get_ticks() - self.end_ticks < INPUT_DELAY_MS:
+         return
+     if event.type == pygame.KEYDOWN:
+         if event.key == pygame.K_ESCAPE:

```

```

        self.quit_requested = True
        elif event.key in (pygame.K_RETURN, pygame.K_KP_ENTER, pygame.K_SPACE):
-         self.reset()
+         self._open_menu()
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
-         self.reset()
+         self._open_menu()
+
+     def _open_menu(self):
+         self.state = "menu"
+         self.menu_index = MENU_OPTIONS.index(self.difficulty)
+
+     def _choose(self, option):
+         if option == "Exit":
+             self.quit_requested = True
+         else:
+             self.start_round(option)
+
+     def _handle_menu_event(self, event):
+         if event.type == pygame.KEYDOWN:
+             if event.key == pygame.K_ESCAPE:
+                 self.quit_requested = True
+             elif event.key in (pygame.K_UP, pygame.K_w):
+                 self.menu_index = (self.menu_index - 1) % len(MENU_OPTIONS)
+             elif event.key in (pygame.K_DOWN, pygame.K_s):
+                 self.menu_index = (self.menu_index + 1) % len(MENU_OPTIONS)
+             elif event.key in (pygame.K_RETURN, pygame.K_KP_ENTER, pygame.K_SPACE):
+                 self._choose(MENU_OPTIONS[self.menu_index])
+             elif event.key in (pygame.K_1, pygame.K_KP1):
+                 self._choose("Easy")
+             elif event.key in (pygame.K_2, pygame.K_KP2):
+                 self._choose("Medium")
+             elif event.key in (pygame.K_3, pygame.K_KP3):
+                 self._choose("Hard")
+             elif event.type == pygame.MOUSEMOTION:
+                 for i, (rect) in enumerate(self.buttons()):
+                     if rect.collidepoint(event.pos):
+                         self.menu_index = i
+             elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+                 for label, rect in self.buttons():
+                     if rect.collidepoint(event.pos):
+                         self._choose(label)
+                         break
+
+     def buttons(self):
+         """(label, rect) for each menu button - used for clicks and drawing."""
+         w, h, gap = 340, 46, 14
+         top = 175
+         out = []
+         for i, label in enumerate(MENU_OPTIONS):
+             rect = pygame.Rect(0, 0, w, h)
+             rect.centerx = self.width // 2
+             rect.top = top + i * (h + gap)
+             out.append((label, rect))
+         return out
+
+     def handle_input(self):
+         if self.state != "playing":
@@ -214,6 +288,8 @@ class GameEngine:
            self._draw_board(screen)
            if self.state == "game_over":
                self._draw_game_over(screen)
+         elif self.state == "menu":
+             self._draw_menu(screen)

    def _draw_board(self, screen):
        screen.fill(DARK)
@@ -230,6 +306,9 @@ class GameEngine:
        timer_text = self.font.render(f"Time: {seconds_left}s", True, color)
        screen.blit(timer_text, (10, 10))

```



```

@@ -48,6 +55,7 @@ class GameEngine:
    self.small_font = pygame.font.SysFont("Arial", 20)

    self.quit_requested = False # main loop exits when this is True
+   self.sounds = SoundBank()
    self.reset()

    # ----- state
@@ -74,6 +82,8 @@ class GameEngine:
    self.finish_time_ms = None
    self.end_elapsed_ms = None
    self.end_ticks = None
+   self.last_bounce_ticks = -BOUNCE_COOLDOWN_MS
+   self.last_tick_second = None

    def _end_round(self, result, elapsed_ms):
        self.state = "game_over"
@@ -83,6 +93,9 @@ class GameEngine:
    self.end_ticks = pygame.time.get_ticks()
    if result == "solved":
        self.finish_time_ms = elapsed_ms
+       self.sounds.play("goal", 0.9)
+   else:
+       self.sounds.play("timeout", 0.9)

    def elapsed_ms(self):
        """Time used this round - frozen once the round is over."""
@@ -111,6 +124,9 @@ class GameEngine:
    def handle_event(self, event):
        # Tilting is driven by the continuous mouse position (handle_input);
        # discrete events are only used on the end screen and the menu.
+       if event.type == pygame.KEYDOWN and event.key == pygame.K_m:
+           self.sounds.toggle_mute()
+           return
        if self.state == "game_over":
            self._handle_game_over_event(event)
        elif self.state == "menu":
@@ -210,13 +226,29 @@ class GameEngine:
        self.marble.x += self.marble.vx
        self.marble.y += self.marble.vy

-       self._resolve_wall_collisions()
+       impact = self._resolve_wall_collisions()
+       self._bounce_sound(impact)
+       self._warning_tick(elapsed)

        gx = self.goal_x - self.marble.x
        gy = self.goal_y - self.marble.y
        if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
            self._end_round("solved", elapsed)

+   def _bounce_sound(self, impact):
+       now = pygame.time.get_ticks()
+       if impact < BOUNCE_MIN_IMPACT or now - self.last_bounce_ticks < BOUNCE_COOLDOWN_MS:
+           return
+       self.last_bounce_ticks = now
+       volume = 0.25 + 0.75 * min(1.0, impact / BOUNCE_FULL_IMPACT)
+       self.sounds.play("bounce", volume)
+
+   def _warning_tick(self, elapsed):
+       seconds_left = math.ceil((self.time_limit_ms - elapsed) / 1000)
+       if 0 < seconds_left <= WARNING_SECONDS and seconds_left != self.last_tick_second:
+           self.last_tick_second = seconds_left
+           self.sounds.play("tick", 0.6)
+
    def _resolve_wall_collisions(self):
        """Circle-vs-rectangle collision (Task 1).

@@ -308,6 +340,9 @@ class GameEngine:

```

```

        diff_text = self.small_font.render(self.difficulty, True, DIFFICULTY_COLORS[self.difficulty])
        screen.blit(diff_text, (self.width - diff_text.get_width() - 12, 14))
+
+     if self.sounds.muted:
+         mute_text = self.small_font.render("muted (M)", True, MUTED)
+         screen.blit(mute_text, (self.width - mute_text.get_width() - 12, 36))

    def _dim(self, screen, alpha=170):
        shade = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
diff --git a/Lab-4/code/game/sounds.py b/Lab-4/code/game/sounds.py
new file mode 100644
index 0000000..c660fb2
--- /dev/null
+++ b/Lab-4/code/game/sounds.py
@@ -0,0 +1,143 @@
+"""Task 4: sound effects.
+
+All sounds are synthesised in code when the game starts, so the project
+needs no audio files and no extra dependencies (no numpy). If the computer
+has no audio device the game still runs - it just stays silent.
+
+Sounds:
+    bounce    - short wooden "tock" when the marble hits a wall (louder for
+                harder hits)
+    goal       - rising arpeggio when the marble reaches the goal
+    tick       - soft tick for each of the last 5 seconds
+    timeout    - descending "wah-wah" when the timer runs out
+"""
+import array
+import math
+import random
+
+import pygame
+
+RATE = 44100
+
+def _envelope(i, n, attack=0.004, decay=None):
+    t = i / RATE
+    a = min(1.0, t / attack) if attack > 0 else 1.0
+    if decay is None:
+        # linear fade-out over the last 20% so notes don't click
+        tail = max(1, int(n * 0.2))
+        d = min(1.0, (n - i) / tail)
+    else:
+        d = math.exp(-t / decay)
+    return a * d
+
+def _tone(freq, seconds, volume=0.5, decay=None, harmonics=((1, 1.0)), vibrato=0.0):
+    n = int(RATE * seconds)
+    out = []
+    for i in range(n):
+        t = i / RATE
+        f = freq * (1 + vibrato * math.sin(2 * math.pi * 6 * t))
+        s = sum(amp * math.sin(2 * math.pi * f * mult * t) for mult, amp in harmonics)
+        out.append(s * volume * _envelope(i, n, decay=decay))
+    return out
+
+def _concat(*parts):
+    out = []
+    for p in parts:
+        out.extend(p)
+    return out
+
+def _make_bounce():
+    rng = random.Random(7)
+    n = int(RATE * 0.07)
+    out = []
+    for i in range(n):

```

```

+         t = i / RATE
+         body = math.sin(2 * math.pi * 260 * t) * math.exp(-t / 0.018)
+         click = (rng.random() * 2 - 1) * math.exp(-t / 0.003)
+         out.append(0.55 * body + 0.25 * click)
+     return out
+
+
+def _make_goal():
+    bright = ((1, 1.0), (2, 0.35), (3, 0.15))
+    notes = [523.25, 659.25, 783.99] # C5 E5 G5
+    parts = [_tone(f, 0.09, 0.35, harmonics=bright) for f in notes]
+    parts.append(_tone(1046.5, 0.45, 0.35, decay=0.18, harmonics=bright)) # C6
+    return _concat(*parts)
+
+
+def _make_tick():
+    return _tone(1400, 0.035, 0.25, decay=0.008)
+
+
+def _make_timeout():
+    brass = ((1, 1.0), (2, 0.5), (3, 0.3), (4, 0.15))
+    return _concat(
+        _tone(392.00, 0.22, 0.32, harmonics=brass), # G4
+        _tone(369.99, 0.22, 0.32, harmonics=brass), # F#4
+        _tone(349.23, 0.22, 0.32, harmonics=brass), # F4
+        _tone(329.63, 0.75, 0.32, harmonics=brass, vibrato=0.012), # E4 (held)
+    )
+
+
+class SoundBank:
+    def __init__(self):
+        self.rate = RATE
+        # mono float samples in [-1, 1]; kept so the sounds can be reused
+        # (e.g. by a recorder) even when there is no audio device
+        self.samples = {
+            "bounce": _make_bounce(),
+            "goal": _make_goal(),
+            "tick": _make_tick(),
+            "timeout": _make_timeout(),
+        }
+        self.muted = False
+        self.sounds = {}
+        self.enabled = self._init_mixer()
+        if self.enabled:
+            for name, data in self.samples.items():
+                try:
+                    self.sounds[name] = pygame.mixer.Sound(buffer=self._to_pcm(data))
+                except pygame.error:
+                    pass
+
+    def _init_mixer(self):
+        try:
+            if pygame.mixer.get_init() is None:
+                pygame.mixer.init(RATE, -16, 1, 512)
+            freq, fmt, channels = pygame.mixer.get_init()
+        except pygame.error:
+            return False
+        if fmt not in (-16, 16): # we only synthesise 16-bit audio
+            return False
+        self._mixer_rate, self._channels = freq, channels
+        return True
+
+    def _to_pcm(self, data):
+        # resample (nearest) if the mixer runs at a different rate
+        if self._mixer_rate != RATE:
+            step = RATE / self._mixer_rate
+            data = [data[int(i * step)]] for i in range(int(len(data) / step))
+        pcm = array.array("h")
+        for s in data:
+            v = int(max(-1.0, min(1.0, s)) * 32000)
+            pcm.extend([v] * self._channels)

```

```

+         return pcm.tobytes()
+
+     def play(self, name, volume=1.0):
+         if self.muted:
+             return
+         snd = self.sounds.get(name)
+         if snd is None:
+             return
+         snd.set_volume(max(0.0, min(1.0, volume)))
+         snd.play()
+
+     def toggle_mute(self):
+         self.muted = not self.muted
+         if self.muted and self.enabled:
+             pygame.mixer.stop()

```

```

diff --git a/Lab-4/code/main.py b/Lab-4/code/main.py
index 85b3c75..e49978d 100644

```

```

--- a/Lab-4/code/main.py

```

```

+++ b/Lab-4/code/main.py

```

```

@@ -2,6 +2,8 @@ import pygame

```

```

    from game.game_engine import GameEngine

```

```

# Initialize pygame/Start application

```

```

+# small audio buffer so sound effects play without noticeable lag (Task 4)

```

```

+pygame.mixer.pre_init(44100, -16, 1, 512)

```

```

pygame.init()

```

```

# Screen dimensions

```